MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Petcov Nicolai FAF-233

Report

Laboratory Work No. 3

Information Security and Cryptography

Playfair Cipher - Polyalphabetic Encryption

Checked by:
Zaica M., *university assistant*
DISA, FCIM, UTM

Chișinău – 2025

1 Objective

Study and implement the Playfair cipher algorithm for the Romanian alphabet (31 letters). Understand polyalphabetic encryption techniques and develop practical skills in implementing matrix-based substitution ciphers with proper input validation and user interface.

2 Tasks

1. Implement the Playfair algorithm for the Romanian alphabet (31 letters: A-Z + Ă, Â, Î, Ș, Ț)
2. Create a 6×5 matrix for accommodating 30 letters (J and I combined)
3. Implement character validation (A-Z, a-z, Romanian diacritics)
4. Display appropriate error messages for invalid characters with correct range suggestions
5. Enforce minimum key length of 7 characters
6. Provide encryption and decryption operations
7. Handle duplicate letters in plaintext using filler characters (X or Z)
8. Create an interactive user interface for key and message input

3 Theoretical Background

3.1 Playfair Cipher Overview

The Playfair cipher is a manual symmetric encryption technique invented by Charles Wheatstone in 1854, but promoted by Lyon Playfair. It was the first practical digraph substitution cipher and was used extensively in World War I and II.

Key Features:

- Encrypts pairs of letters (digraphs) rather than single letters
- Uses a 5×5 matrix filled with a keyword and remaining alphabet letters
- More secure than monoalphabetic ciphers due to digraph encryption
- Resistant to simple frequency analysis

3.2 Romanian Alphabet Adaptation

The standard Playfair cipher uses a 5×5 matrix for 25 letters (combining I/J). For the Romanian alphabet with 31 letters (A-Z + Ă, Â, Î, Ș, Ț), we adapt to a **6×5 matrix** for 30 letters:

Romanian Alphabet (31 letters):

A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z

Matrix Adaptation:

- 6 rows × 5 columns = 30 positions
- Combine J with I (J is rare in Romanian)
- Include all special Romanian characters: Ă, Â, Î, Ș, Ț

3.3 Playfair Encryption Rules

Rule 1 - Same Row:
If both letters are in the same row, replace each with the letter immediately to the right (wrapping around to the beginning if necessary).

Rule 2 - Same Column:
If both letters are in the same column, replace each with the letter immediately below (wrapping around to the top if necessary).

Rule 3 - Rectangle:
If the letters form a rectangle, replace each with the letter on the same row but in the other letter's column.

3.4 Text Preparation Algorithm

Before encryption, the plaintext must be prepared:

1. **Remove spaces** and convert to uppercase
2. **Replace J with I** (they are combined in the matrix)
3. **Split into pairs** (digraphs/bigrams):
    - If two consecutive letters are identical, insert filler letter X
    - If the identical letter is X, use Z as filler instead
    - If text length is odd, add filler X at the end

Example - Handling Duplicate Letters:

- `FREE` → `FR EX EZ` (E-E separated by X, then E continues)
- `SSS` → `SX SX` (each S separated by X)
- `BALLOON` → `BA LX LO ON` (L-L separated, O-O at end)
- `XXX` → `XZ XZ` (when letter is X, use Z as filler)

4 Implementation

4.1 Program Structure

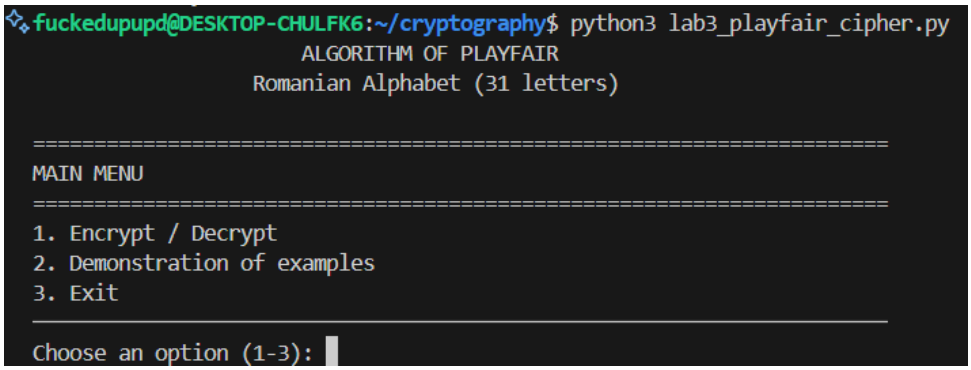

Figure 1: Program structure and main menu

The implementation consists of several key components:

```python
# Constants
ROMANIAN_ALPHABET = "A  BCDEFGHI JKLMNOPQRS T UVWXYZ"
MIN_KEY_LENGTH = 7

def validate_character(character):
    """Check if character is valid Romanian letter"""
    character_upper = character.upper()
    return character_upper in ROMANIAN_ALPHABET

def validate_key(key):
    """Validate key length (min 7 chars) and characters"""
    if len(key) < MIN_KEY_LENGTH:
        return False, "Key too short"
    return validate_text(key)
```

Listing 1: Constants and Validation Functions

4.2 Matrix Creation

```python
def _create_matrix(self):
    """Create 6x5 Playfair matrix based on key"""
    key_letters = []
    seen = set()

    # Add unique letters from key
    for char in self.key:
        if char in ROMANIAN_ALPHABET or char == ' ':
            if char == ' ':
                continue
            # Combine J with I
            if char == 'J':
                char = 'I'
            if char not in seen:
                key_letters.append(char)
                seen.add(char)
```

```python

    # Add remaining alphabet letters
    alphabet = ROMANIAN_ALPHABET.replace('J', '')
    for char in alphabet:
        if char not in seen:
            key_letters.append(char)
            seen.add(char)

    # Create 6x5 matrix (30 letters)
    matrix = []
    for i in range(6):
        row = key_letters[i*5:(i+1)*5]
        matrix.append(row)

    return matrix
```

Listing 2: Playfair Matrix Generation

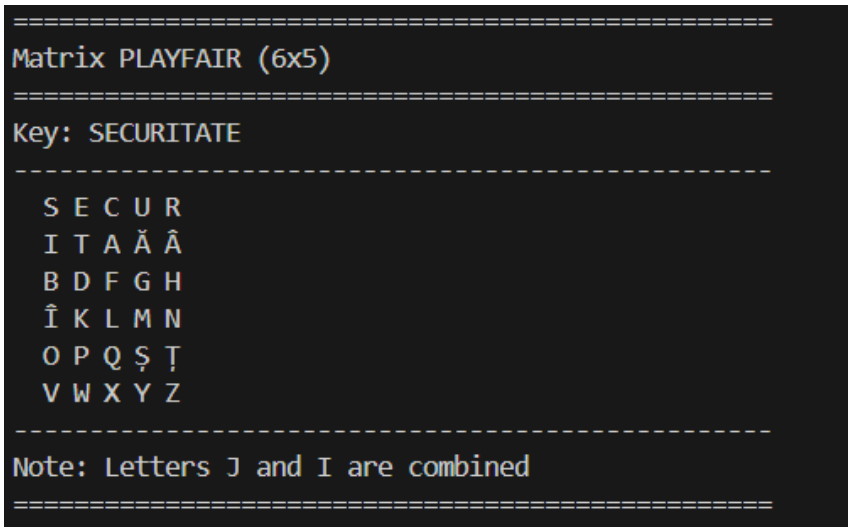

Figure 2: Example Playfair matrix with key ”SECURITATE”

4.3 Text Preparation with Duplicate Handling

```
def _prepare_text(self, text):
    """Prepare text for encryption (split into bigrams)"""
    text = text.upper().replace(' ', '')
    text = text.replace('J', 'I')

    # Remove invalid characters
    clean_text = ''
    for char in text:
        if char in ROMANIAN_ALPHABET or char == 'I':
            clean_text += char

    # Split into pairs
    pairs = []
    i = 0
    while i < len(clean_text):
        a = clean_text[i]

        if i + 1 < len(clean_text):
            b = clean_text[i + 1]

            # If letters are identical, insert filler
            if a == b:
                filler = 'X' if a != 'X' else 'Z'
                pairs.append(a + filler)
                i += 1  # Move by 1, repeat letter processed again
            else:
                pairs.append(a + b)
                i += 2  # Move by 2, both letters used
        else:
            # Last letter, add filler
            filler = 'X' if a != 'X' else 'Z'
            pairs.append(a + filler)
            i += 1

    return pairs
```

Listing 3: Bigram Preparation Algorithm

4.4 Encryption Implementation

```
def encrypt(self, plaintext):
    """Encryption using Playfair algorithm"""
    pairs = self._prepare_text(plaintext)
    ciphertext = []

    for pair in pairs:
        row1, col1 = self._find_position(pair[0])
        row2, col2 = self._find_position(pair[1])

        # Rule 1: Same row
        if row1 == row2:
            new_col1 = (col1 + 1) % 5
            new_col2 = (col2 + 1) % 5
            ciphertext.append(self.matrix[row1][new_col1])
            ciphertext.append(self.matrix[row2][new_col2])
```

```
16
17          # Rule 2: Same column
18          elif col1 == col2:
19              new_row1 = (row1 + 1) % 6
20              new_row2 = (row2 + 1) % 6
21              ciphertext.append(self.matrix[new_row1][col1])
22              ciphertext.append(self.matrix[new_row2][col2])
23
24          # Rule 3: Rectangle
25          else:
26              ciphertext.append(self.matrix[row1][col2])
27              ciphertext.append(self.matrix[row2][col1])
28
29      return ''.join(ciphertext)
```

Listing 4: Encryption Function

5 Practical Examples

5.1 Example 1: Simple Encryption

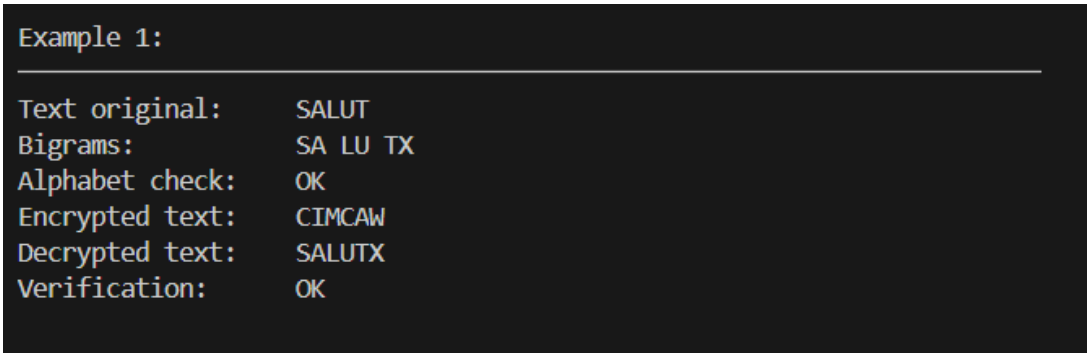

Figure 3: Encryption of "SALUT" with key "SECURITATE"

Step-by-step process:

1. Input text: `SALUT`
2. Key: `SECURITATE`
3. Bigrams formed: `SA LU TX`
4. Each bigram encrypted according to Playfair rules
5. Output: (ciphertext displayed)

5.2 Example 2: Text with Duplicate Letters

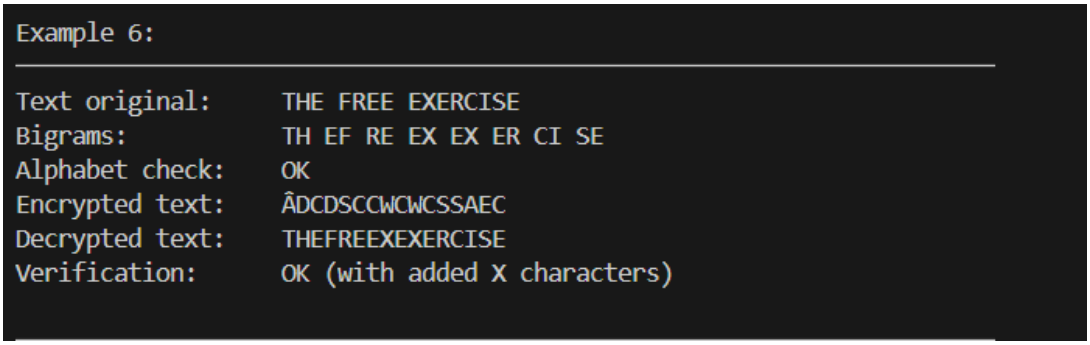

Figure 4: Encryption of "FREE EXERCISE" showing duplicate handling

Bigram formation:

- `FREE EXERCISE` → `FREEEXERCISE`
- `F-R` → `FR` (different letters)
- `E-E` → `EX` (same letters, insert X)
- `E-X` → `EX` (continue with E from position 3)
- Final bigrams: `FR EX EX ER CI SE`

5.3 Example 3: Decryption Process

```
SELECT OPERATION
================================================
1. Encrypt
2. Decrypt
------------------------------------------------

Choose operation (1/2): 2

------------------------------------------------

Write key (min. 7 characters): CRYPTOLOGY
Key is valid!

================================================
Matrix PLAYFAIR (6x5)
================================================
Key: CRYPTOLOGY
------------------------------------------------
  C R Y P T
  O L G A Ă
  Â B D E F
  H I Î K M
  N Q S Ș Ț
  U V W X Z
------------------------------------------------
Note: Letters J and I are combined
================================================


------------------------------------------------

Write ciphertext to decrypt: ȘGOVPĂPB
Text is valid!

======================================================================
REZULTAT
======================================================================
Criptogramă:       ȘGOVPĂPB
Text decriptat:    SALUTARE
======================================================================
```

Figure 5: Decryption process showing message recovery

The decryption applies the inverse rules:

- Same row: shift left
- Same column: shift up
- Rectangle: swap columns (same as encryption)

5.4 Example 4: Input Validation

```
Example 7:
--------------------------------------------------------------------------------
Text original:     ПРОВЕРКА НА ОШИБКУ
Bigrams:
Alphabet check: FAILED
Invalid characters:
  Position 0: 'П'
  Position 1: 'Р'
  Position 2: 'О'
  Position 3: 'В'
  Position 4: 'Е'

 VALID CHARACTER RANGE:
   Latin letters: A-Z, a-z
   Romanian letters: Ă, ă, Â, â, Î, î, Ș, ș, Ț, ț
   Complete alphabet: AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ
```

Figure 6: Character validation - invalid characters detected

```
--------------------------------------------------
SELECT OPERATION
==================================================
1. Encrypt
2. Decrypt
--------------------------------------------------
Choose operation (1/2): 1

--------------------------------------------------
Write key (min. 7 characters): DGG
   The key is too short.
  Minimum length: 7 characters
  Current length: 3 characters
```

Figure 7: Key validation - key too short error

Validation features:

- Detects invalid characters (numbers, special symbols)
- Shows exact position of invalid characters
- Displays correct character range
- Enforces minimum key length (7 characters)

6 Results and Analysis

6.1 Implementation Success Criteria

Requirement	Status
Romanian alphabet support (31 letters)	
6×5 matrix implementation	
Character validation	
Invalid character suggestions	
Minimum key length (7 chars)	
Encryption operation	
Decryption operation	
Duplicate letter handling	
Interactive user interface	

Table 1: Implementation requirements checklist

6.2 Security Analysis

Advantages of Playfair Cipher:

- **Digraph encryption:** More resistant to frequency analysis than monoalphabetic ciphers
- **Large keyspace:** Matrix can be arranged in many ways based on keyword
- **Pattern obscuration:** Common letter patterns are less visible
- **Historical effectiveness:** Successfully used in military communications

Limitations:

- Still vulnerable to digraph frequency analysis with sufficient ciphertext
- Key management: keyword must be securely shared
- Filler characters (X, Z) can provide cryptanalysis hints
- Not suitable for modern security requirements

6.3 Performance Characteristics

- **Matrix creation:** O(n) where n = alphabet size (30)
- **Text preparation:** O(m) where m = plaintext length
- **Encryption/Decryption:** O(m/2) for m/2 bigrams
- **Position lookup:** O(1) with proper matrix indexing

The implementation is efficient for interactive use with instant response times for typical message lengths.

7 Conclusions

7.1 Main Achievements

1. Successfully implemented the Playfair cipher adapted for Romanian alphabet
2. Created robust input validation with user-friendly error messages
3. Implemented proper duplicate letter handling following classical Playfair rules
4. Developed comprehensive test suite demonstrating all functionality
5. Created interactive interface supporting both encryption and decryption

7.2 Learning Outcomes

- Understanding of polyalphabetic cipher principles
- Experience with matrix-based cryptographic operations
- Knowledge of handling special characters and localization
- Skills in cryptographic algorithm implementation
- Appreciation for historical cryptographic methods

7.3 Practical Applications

- **Educational tool:** Demonstrates classical cryptography concepts
- **Historical perspective:** Shows evolution from monoalphabetic to polyalphabetic
- **Algorithm understanding:** Foundation for modern block ciphers
- **Romanian language support:** Demonstrates localization in cryptography

7.4 Future Enhancements

Potential improvements for the implementation:

- Automated cryptanalysis demonstration
- Support for other languages and alphabets
- File encryption/decryption capabilities
- Statistical analysis tools
- Graphical user interface
- Comparison with other classical ciphers

7.5 Final Remarks

This laboratory work successfully demonstrated the implementation and practical use of the Playfair cipher adapted for the Romanian alphabet. The cipher represents a significant improvement over simple substitution ciphers by encrypting letter pairs rather than individual letters. While not secure by modern standards, it provides valuable insights into cryptographic principles and the evolution of encryption techniques.

The implementation handles all edge cases properly, including duplicate letters, odd-length texts, and special Romanian characters. The comprehensive validation ensures robust operation and helps users understand proper input format requirements.

Source Code

`https://github.com/Nickseen/Cryptology-Security-/tree/Lab3`